Information Retrieval and Extraction
Assignment – 1
IIITH – Monsoon – 2025
2024900005

1. Introduction:

This activity involves building a custom information retrieval (IR) system from scratch to understand how indexing and querying work internally. The system, **SelfIndex**, is compared with **Elasticsearch (ESIndex)** to study differences in performance and architecture.

It includes the implementation of key components such as inverted indexes, postings lists, and skip pointers, along with experiments on ranking methods like word frequency and TF-IDF. Two query processing strategies **Term-at-a-Time (TAA)** and **Document-at-a-Time (DAA)** are evaluated to analyze indexing time, query latency, and memory usage.

The Wikipedia **(20231101.en)** dataset from Hugging Face is used after applying preprocessing steps.

Source Code: https://github.com/darmojud/IRE/tree/Assignment1

2. System Description:

The experiments are conducted using two main indexing systems **ESIndex** and **SelfIndex**

2.1.ESIndex

- Uses **Elasticsearch** as the baseline reference system.
- Implementation done in `src/models/ESIndex.py`.
- Preprocessing Implementation is in `src/core/preprocessor.py`.
- Performs text preprocessing steps identical to SelfIndex:
    - Tokenization
    - Stemming
    - Stopword removal
- Documents are **bulk indexed** into Elasticsearch after preprocessing.
- Executes queries using `query_string` which can handle Boolean processing internally.
- Ensures both **ESIndex** and **SelfIndex** work on **comparable term spaces** and **query structures**.

2.2.SelfIndex

- A **fully custom, modular search engine** implemented in **Python**.
- Implementation present in `src/models/SelfIndex_v2.py`
- Supports multiple configurations with different parameters `xyzqi`.
- Each parameter controls a specific design choice:

Parameter	Enum Used	Description	Configuration Options
x (IndexInfo)	`IndexInfo`	Type of information stored in postings	`BOOLEAN` – (doc_id, [pos]) `WORDCOUNT` – (doc_id, freq) `TFIDF` – (doc_id, score)
y (DataStore)	`DataStore`	Backend used for storing the index	`CUSTOM` – Local JSON/pickle objects `DB1` – RocksDB key-value store `DB2` – Redis in-memory store
z (Compression)	`Compression`	Compression technique for postings lists	`NONE` – No compression `CODE` – Custom integer encoding (e.g., delta/gap) `CLIB` – Off-the-shelf compression library (e.g., Snappy)
q (QueryProc)	`QueryProc`	Query processing method	`TERMatat` – Term-at-a-Time (TAA) `DOCatat` – Document-at-a-Time (DAA)
i (Optimization)	`Optimizations`	Index-level or query-time optimizations	`Null` – No optimization `Skipping` – Skip pointers at √N intervals `Thresholding` – Early pruning using threshold values `EarlyStopping` – Stop when relevance threshold reached

3. Experimental Setup:

3.1.Metrics:

- `Metric - A`: System response time (latency) across a query set with p95 and p99 (95th and 99th percentiles). Create a set that is diverse enough by probing an LLM and justify how it captures various required system properties.
- `Metric - B`: System throughput in queries/second for read or write operations or a mix of both as appropriate for the activity.
- `Mertric - C`: Memory footprint of the system.
- `Mertic - D`: Functional metrics like precision, recall or ranking measures.

3.2.Query Set:

The queries are divided into **six categories** based on complexity and query type:

Category	Count	Description
Single-Term Queries	3	Simple keyword lookups testing basic term retrieval and postings access.
Basic Boolean Queries	4	Tests Boolean operator handling (`AND`, `OR`, `NOT`) and result set merging.
Phrase Queries	3	Evaluates positional indexing and phrase-level matching.
Mixed Queries	4	Combines phrases and Boolean logic to test hybrid retrieval.
Grouped Queries	4	Assesses query tree parsing and grouped logical evaluation.
Nested Queries	4	Exercises deep parsing and nested evaluation performance.

Category	Count	Description
Edge/Tricky Queries	3	Includes ambiguous or corner cases to stress the parser and query optimizer.

3.3.Configuration Setup:

Each experimental configuration is represented as a JSON object containing the following fields:

- **index_type** – Identifies the indexing system being used (e.g., `"self"`).
- **info** – Specifies the indexing strategy, as defined in the `IndexInfo` enum (`BOOLEAN`, `TFIDF`, `WORDCOUNT`).
- **dstore** – Indicates the data storage backend, corresponding to the `DataStore` enum (`CUSTOM`, `DB1`, `DB2`).
- **compr** – Denotes the compression method applied, based on the `Compression` enum (`NONE`, `CODE`, `CLIB`).
- **qproc** – Represents the query processing approach, defined by the `QueryProc` enum (`TERMatat`, `DOCatat`).
- **optim** – Describes any optimization technique used, as listed in the `Optimizations` enum (`Null`, `Skipping`, `Thresholding`, `EarlyStopping`).

Example:

```
[{
  "index_type": "self",
  "info": "BOOLEAN",
  "dstore": "CUSTOM",
  "compr": "NONE",
  "qproc": "TERMatat",
  "optim": "Null"
}]
```

3.4.Logging and output:

Two types of log files are generated to record experimental outcomes — one for indexing performance and another for query performance.

1. **`index_metrics.json`** – Captures metrics related to the indexing phase.

   Each entry in this log records details such as index type, configuration parameters, performance statistics, and resource utilization.

2. **`query_latency_log.csv`** - records detailed query execution metrics for each experimental configuration.

   Each row corresponds to one query execution under a specific setup and includes the following fields:

Field	Description
timestamp	The date and time when the query was executed.
index_type	Indicates which indexing system was used (e.g., `self`, `es`).
info	Type of information stored, corresponding to the `IndexInfo` enum (`BOOLEAN`, `WORDCOUNT`, `TFIDF`).
dstore	Datastore backend used, as defined in `DataStore` (`CUSTOM`, `DB1`, `DB2`).
qproc	Query processing mode, from `QueryProc` (`TERMatat`, `DOCatat`).
compr	Compression method applied, as per `Compression` (`NONE`, `CODE`, `CLIB`).
optim	Optimization strategy used, from `Optimizations` (`Null`, `Skipping`, `Thresholding`, `EarlyStopping`).
query	The actual query string executed (e.g., `"america AND washington"`).
query_type	Categorization of the query (e.g., `Single-Term`, `Boolean`, `Phrase`, `Mixed`, `Grouped`, `Nested`, `Edge`).
run_id	Unique identifier for the experimental run or configuration.
latency_ms	Time taken to execute the query (in milliseconds).
current_mem_kb	Memory usage at the time of query execution (in kilobytes).
peak_mem_kb	Peak memory consumption observed during query execution (in kilobytes).

4. Results & Analysis

4.1. Preprocessing results

Wikipedia dataset after preprocessing, performed on a subset of 1,000 documents:

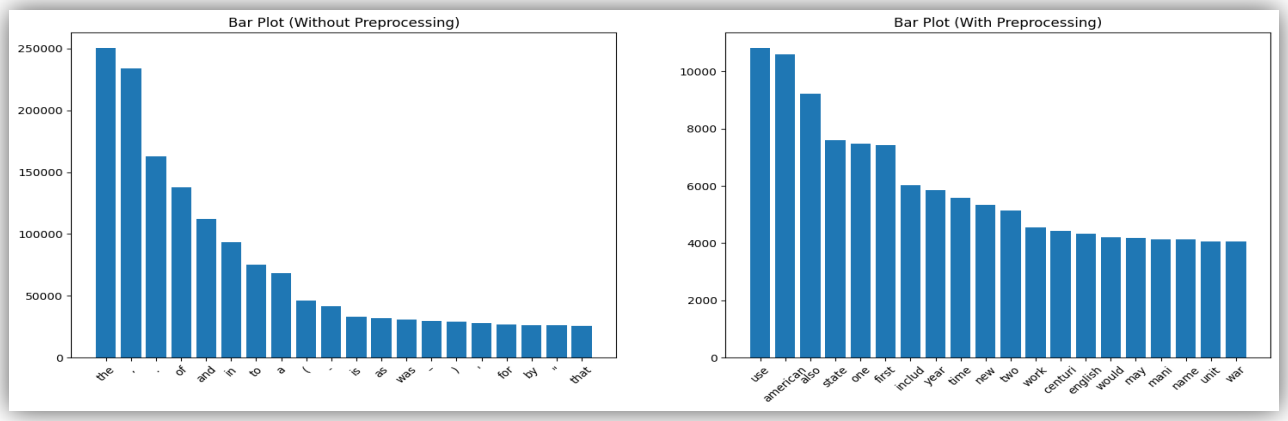

Fig a. Top word frequencies

Fig b. Visual word prominence

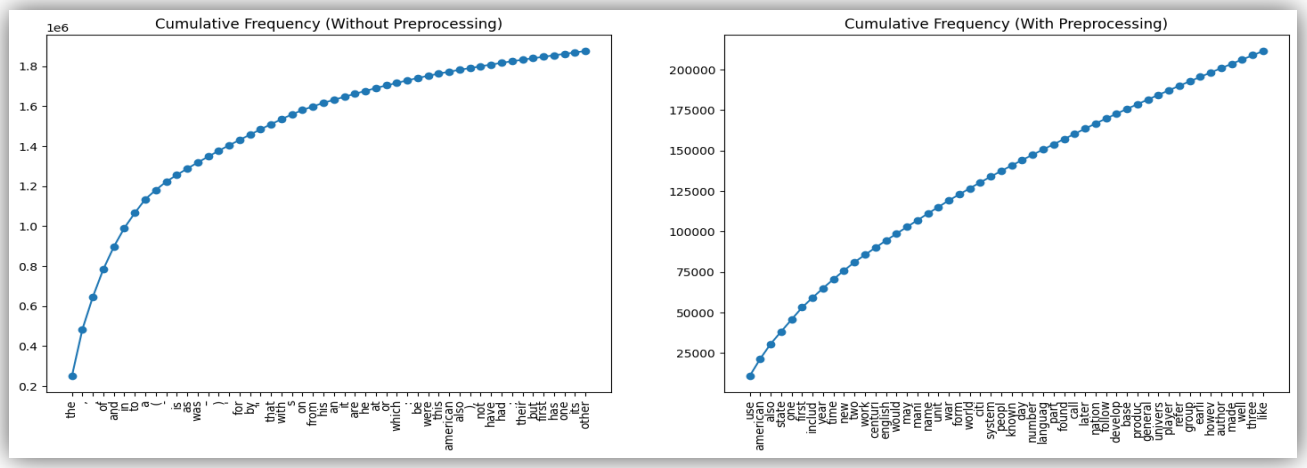

Fig c. Cumulative distribution

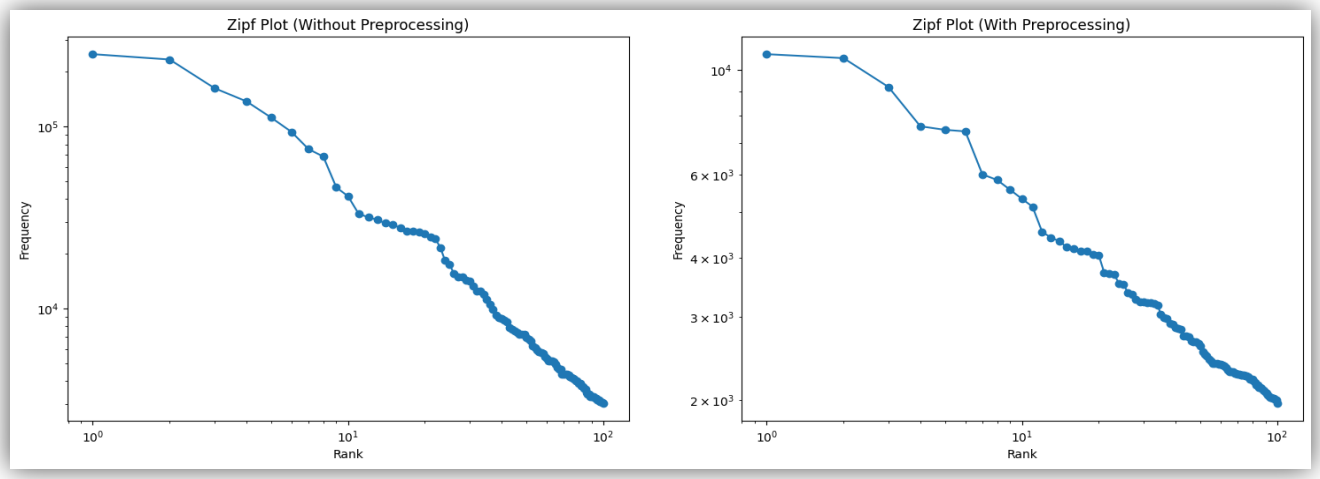

Fig d. Log-rank vs frequency

4.2. Query Latencies on Elastic Search

- Nested and Grouped queries have the highest average latencies (~4 ms and ~3.6 ms), indicating they are the slowest to process.
- Phrase queries are the fastest overall, with an average latency of only ~1.3 ms.

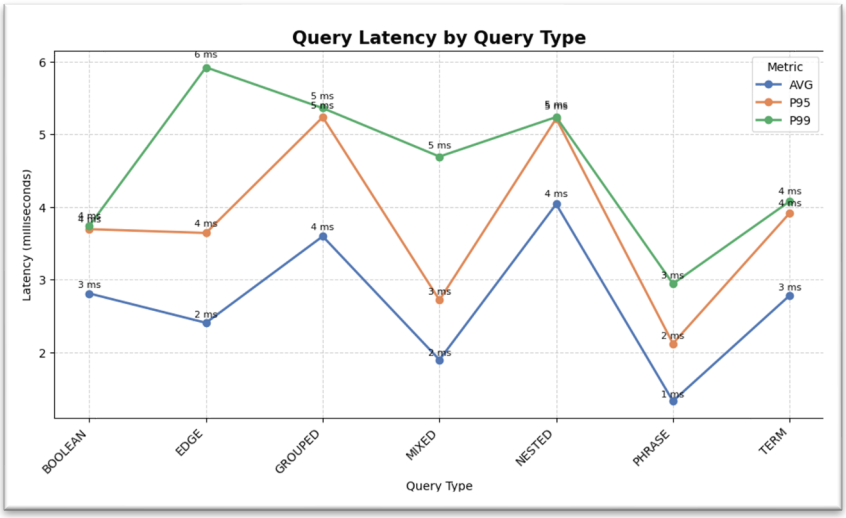

Fig e. ES-Query Latency vs Query Type

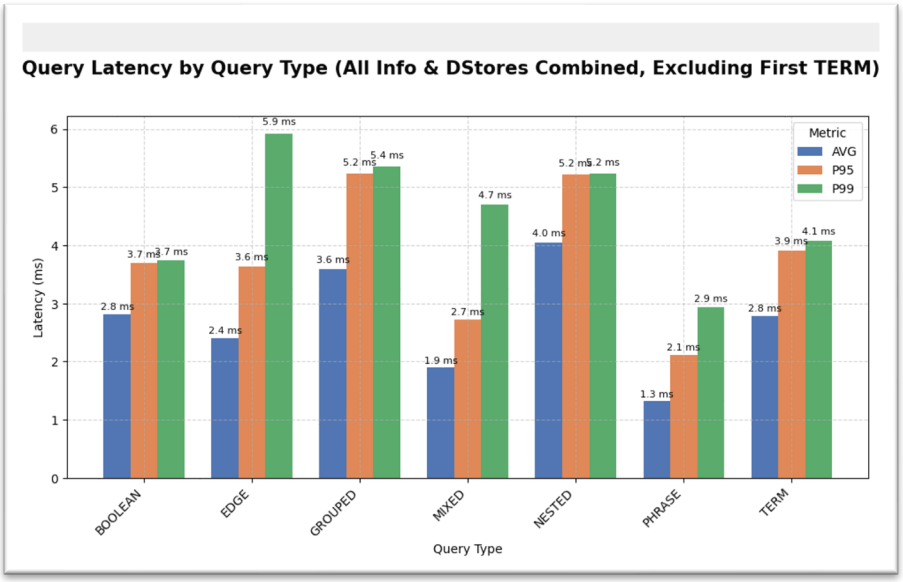

Fig f. ES-Query Latency vs Query Type – Bar graph

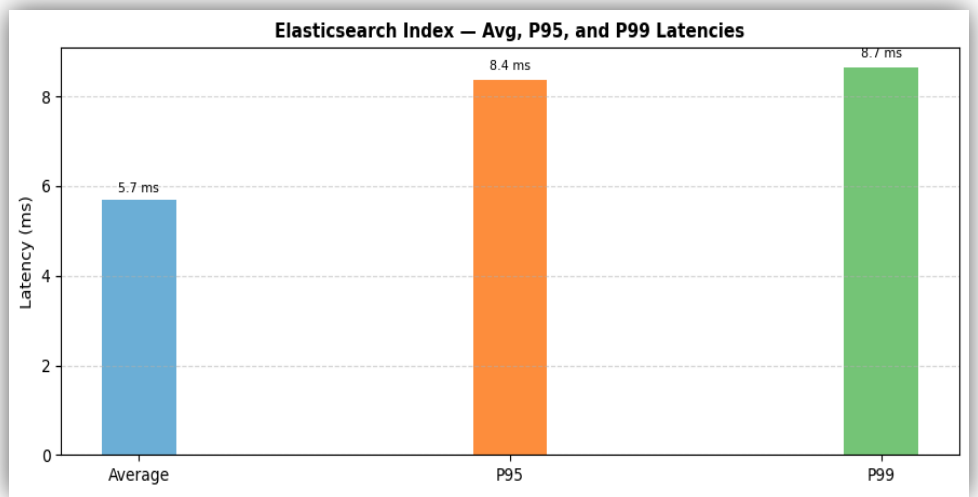

Fig g. ES - Avg, P95, P99 Latencies

- Edge and Mixed queries show higher variability - their p99 values (≈5.9 ms and 4.7 ms) are much larger than their averages.
- Boolean and Term queries have moderate, consistent latencies around 2.8–3 ms with small percentile gaps.
- The results suggest query complexity directly affects latency - deeper or more nested operations cause higher response times.

4.3. Plot C: Memory Footprint of the System Across Different Information Types (x = 1, 2, 3)

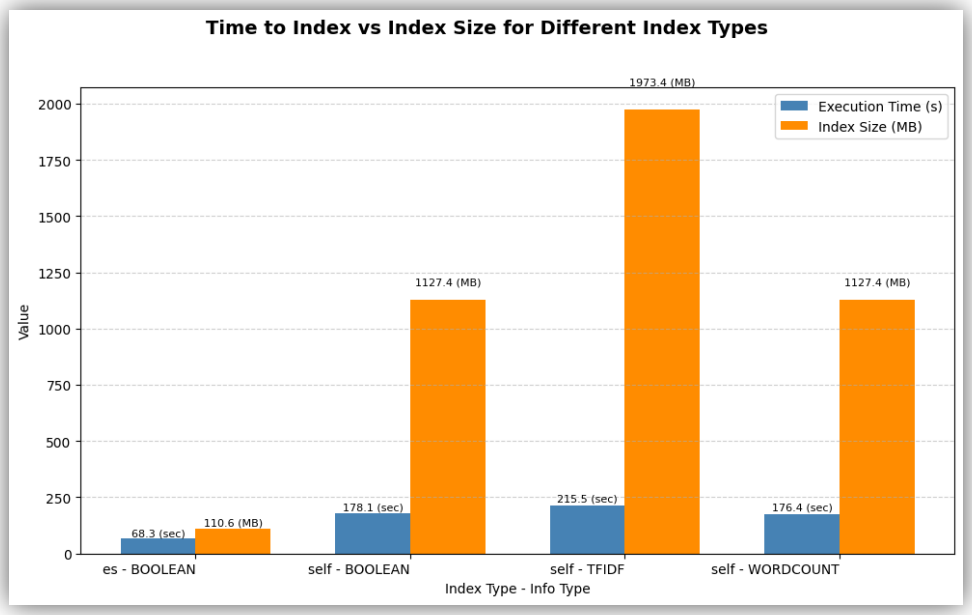

Fig h. Time to Index vs Index Size for different Info types

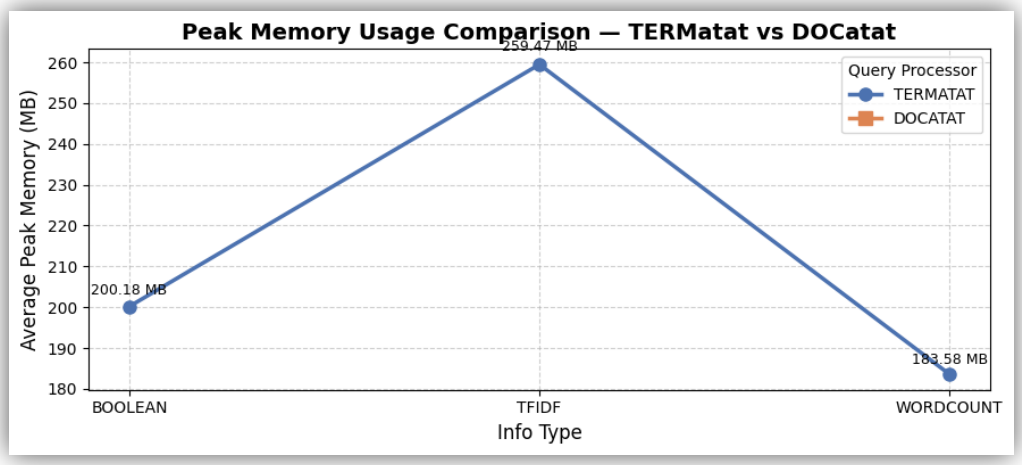

Fig i. Peak Memory Usage Comparison

We can see that the Elasticsearch index is much faster and smaller, whereas self-built indexes take longer and consume more storage.
Average peak memory usage varies by indexing method, with **TFIDF** consuming the most memory.

4.4. Plot A: System Response Time (Latency) Comparison Across Different Datastore Choices (y = 1, 2)

Why Redis and LMDB??

- Redis is an in-memory key–value data store known for its lightning-fast speed, rich data structures, and support for distributed caching and real-time applications. However, it relies heavily on RAM, making it less suitable for very large datasets or high durability requirements.

- LMDB is a lightweight, disk-based, memory-mapped database offering excellent read performance, strong ACID compliance, and minimal overhead. Its main limitation is single-writer concurrency and lack of networked access.

NOTE: Excluding the first term as the loading index takes longer time in custom datastore.

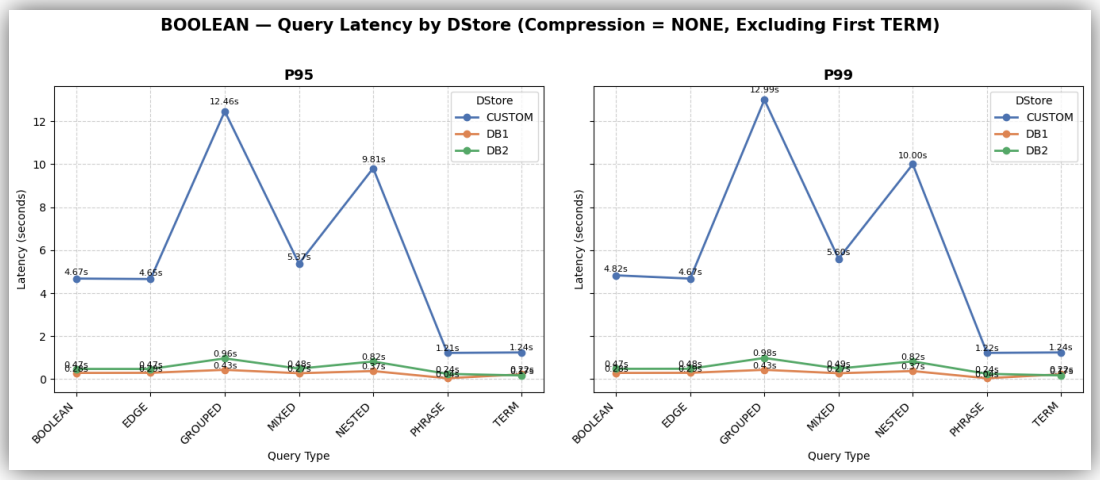

Fig j. SelfIndex- Boolean - Query Latency vs DStore

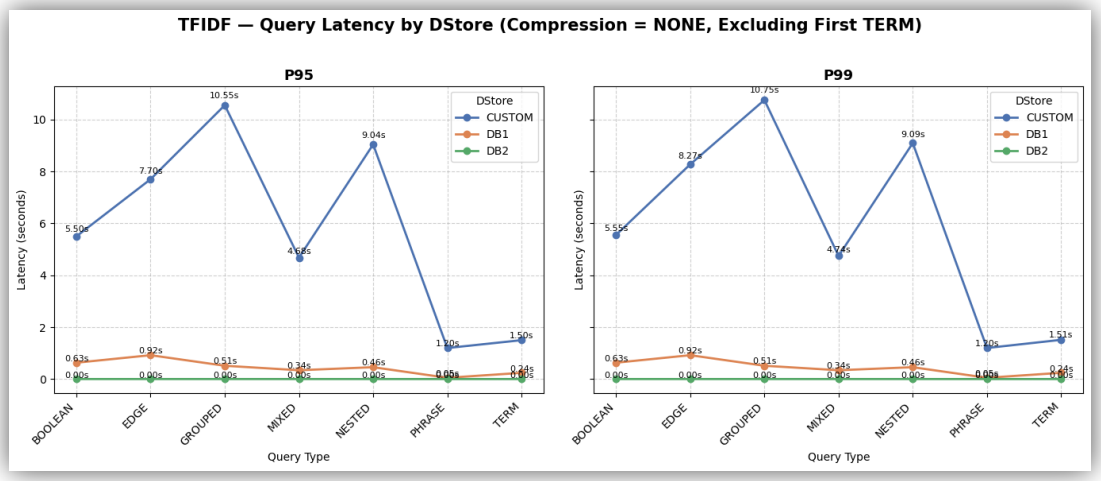

Fig k. SelfIndex- TFIDF - Query Latency vs DStore

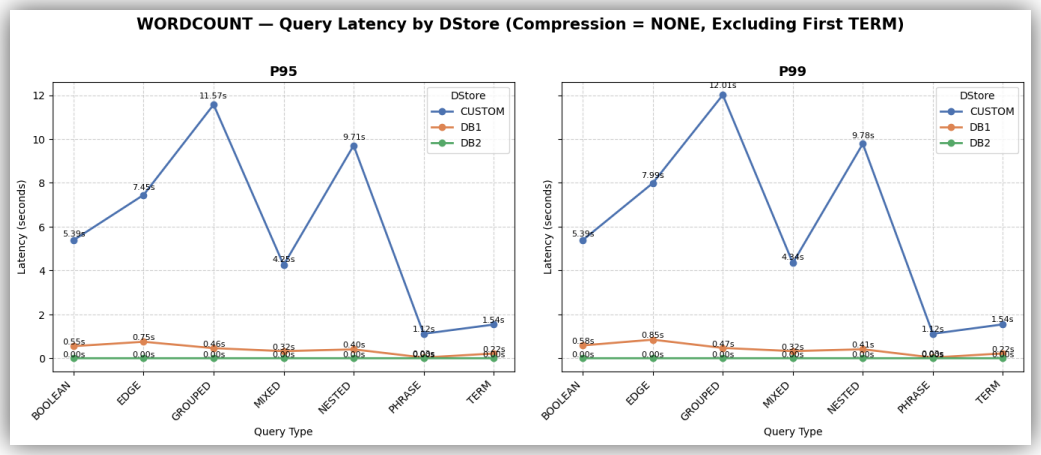

Fig l. SelfIndex- WORDCOUNT - Query Latency vs DStore

4.5. Plot AB: Comparison of System Latency and Throughput for Different Compression Methods (z = 1, 2)

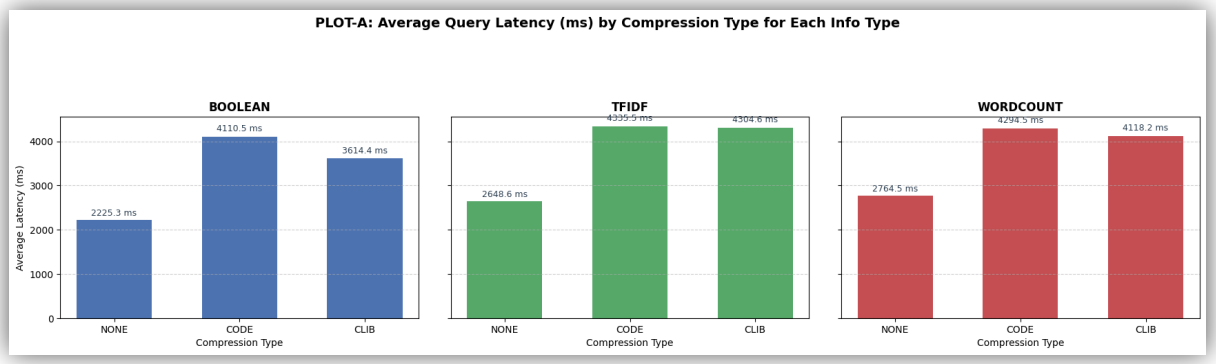

Fig m. SelfIndex – Avg Query Latency vs Compression Type

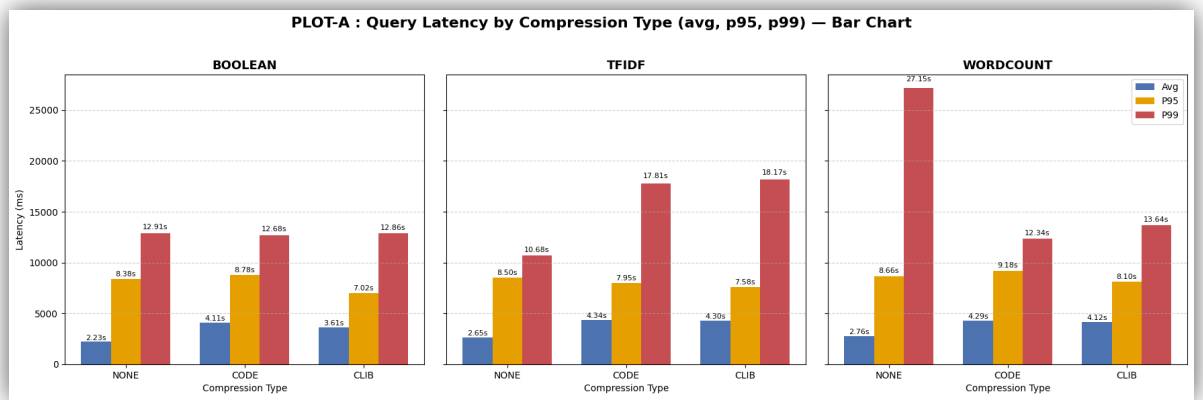

Fig n. SelfIndex – Avg, P95, P99 Query Latency vs Compression Type

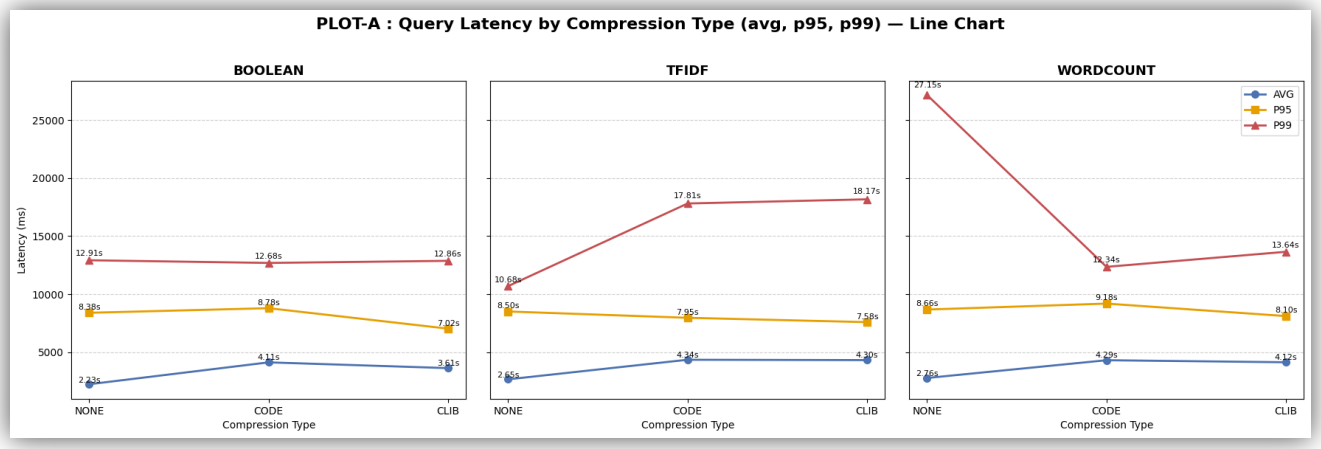

Fig o. SelfIndex – Avg, P95, P99 Query Latency vs Compression Type – Line chart

The above plots indicates the **performance impact** of different retrieval and compression methods on query speed:

- Uncompressed indices (NONE) generally have lower average latency, meaning they respond faster on average.
- However, their p95 and p99 latencies are high, showing they sometimes suffer from large slowdowns (less consistent).
- Compressed indices (CLIB, CODE) show more stable high-percentile performance, meaning fewer extreme slow queries.
- Across retrieval methods, BOOLEAN queries tend to be fastest overall, while TFIDF and WORDCOUNT are slightly slower due to extra scoring computations

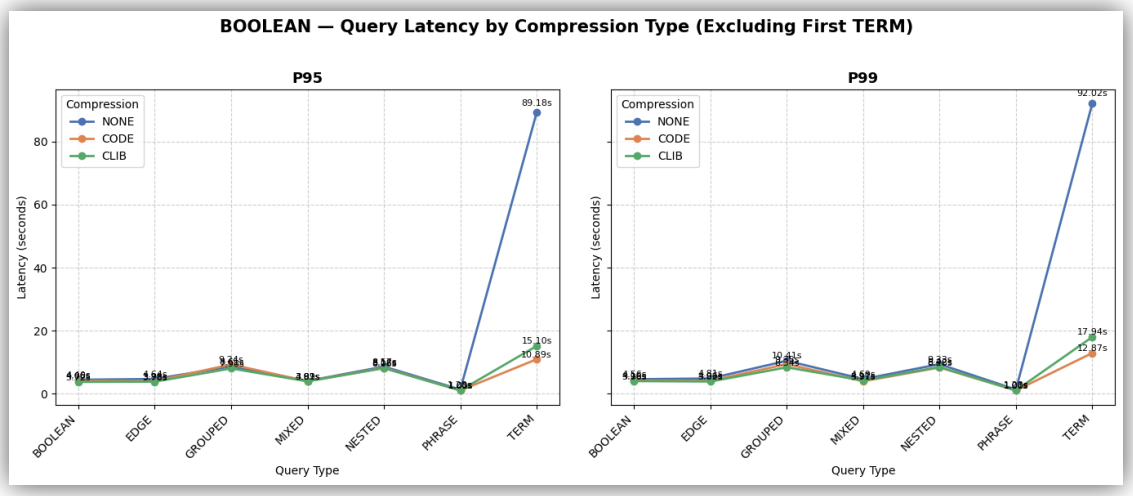

Fig p. SelfIndex – Boolean - P95, P99 Query Latency vs Compression Type

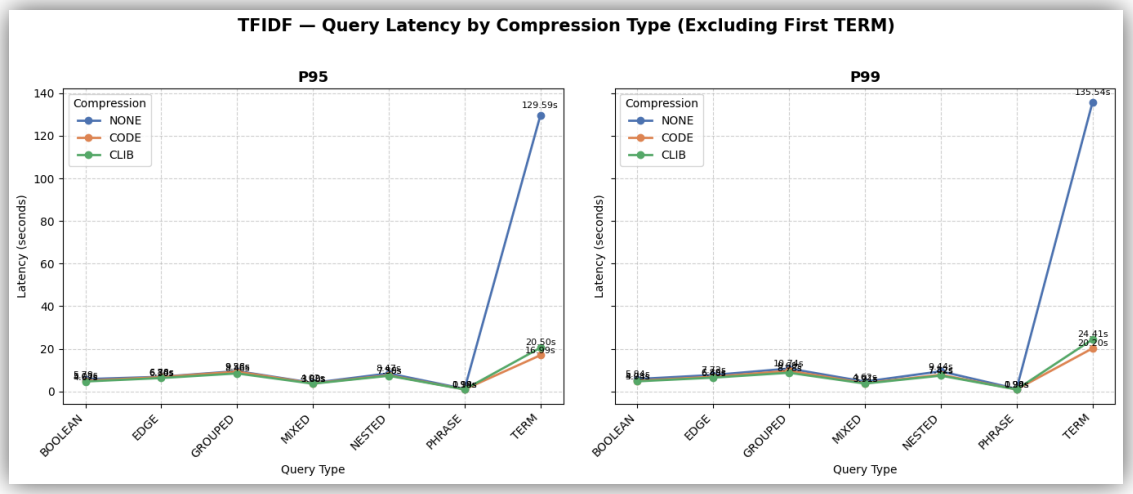

Fig q. SelfIndex – TFIDF - P95, P99 Query Latency vs Compression Type

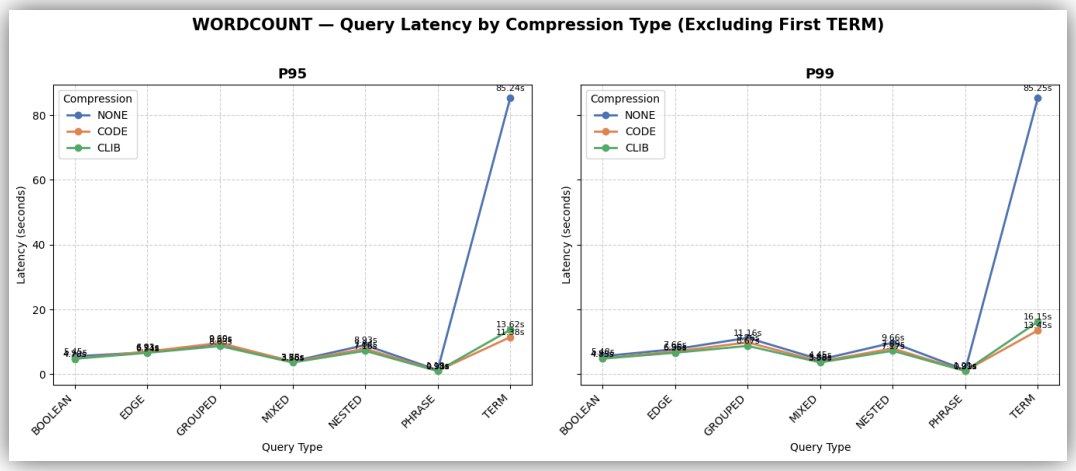

Fig r. SelfIndex – WORDCOUNT - P95, P99 Query Latency vs Compression Type

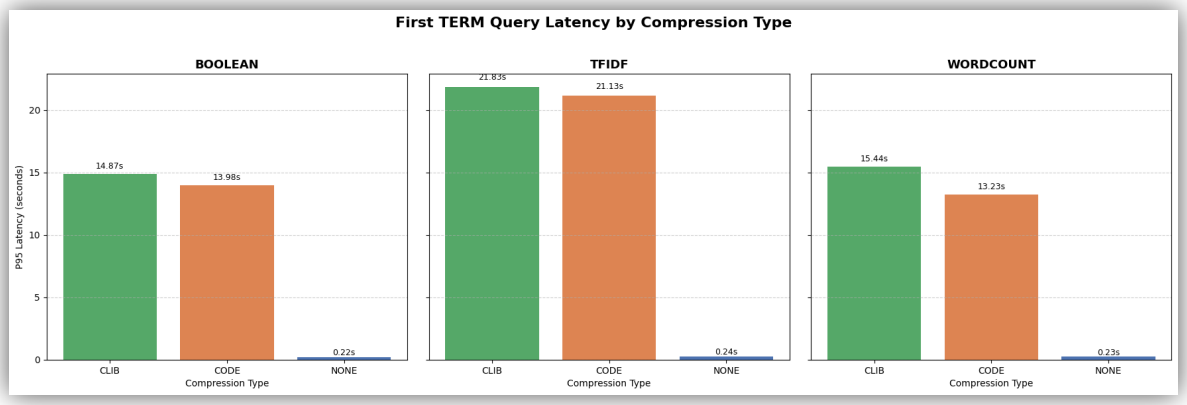

Fig s. SelfIndex – Query Latency vs Compression Type – For First term

- CLIB and CODE compression slightly increase average latency but ensure more stable tail performance.

- Uncompressed (NONE) data gives faster averages but suffers from severe p99 spikes.
- TERM queries are the slowest, while BOOLEAN and PHRASE queries are the fastest.
- TFIDF is slower than BOOLEAN and WORDCOUNT due to additional scoring overhead.
- Overall, compression improves consistency, whereas no compression favors raw speed.

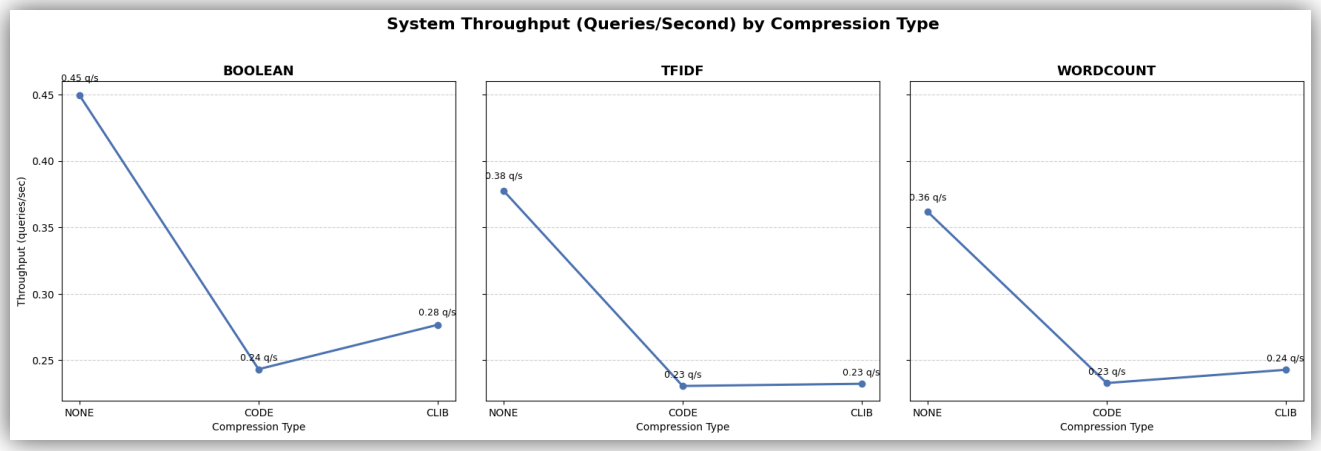

Fig t. SelfIndex – System Throughput

- When compression is NONE, average latency is lowest (around 2200–2700 ms), and throughput is highest (0.36–0.45 QPS).
- When compression is CLIB or CODE, latency increases (around 3600–4300 ms), and throughput drops (0.23–0.27 QPS).
- Compression introduces some overhead during query processing, as data must be decompressed before being read.
- Uncompressed indices (NONE) respond faster but use more disk/memory space.
- BOOLEAN queries are the most efficient overall, while TFIDF and WORDCOUNT take slightly longer due to extra scoring and term frequency calculations.

There's a clear trade-off between storage efficiency and performance: compression saves space but reduces throughput slightly, while uncompressed data gives better speed and responsiveness

4.6. Plot A: Impact of Index-Level Optimizations (i = 0/1) using Skip Pointers on Query Latency

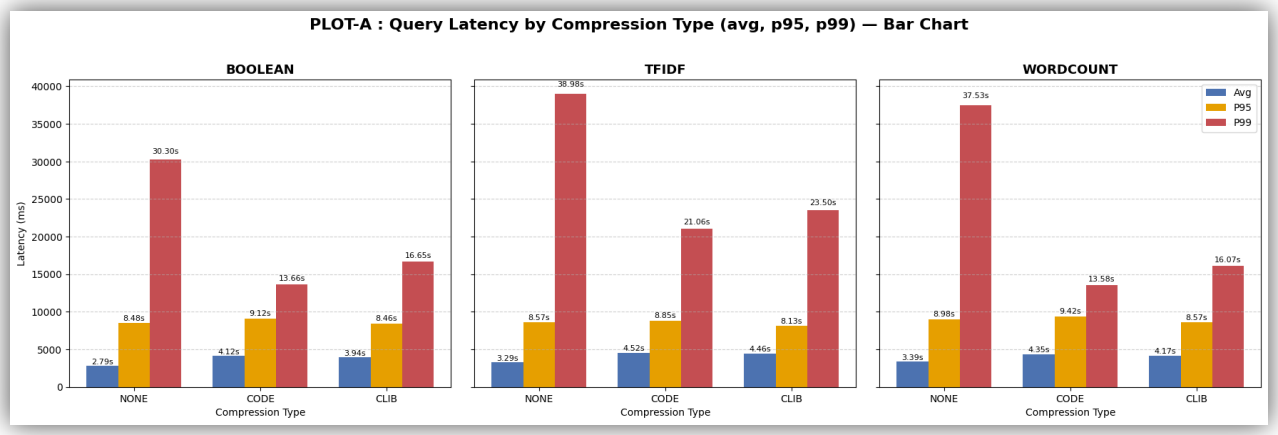

Fig u. SelfIndex – Avg, P95, P99 Query Latency vs Compression Type

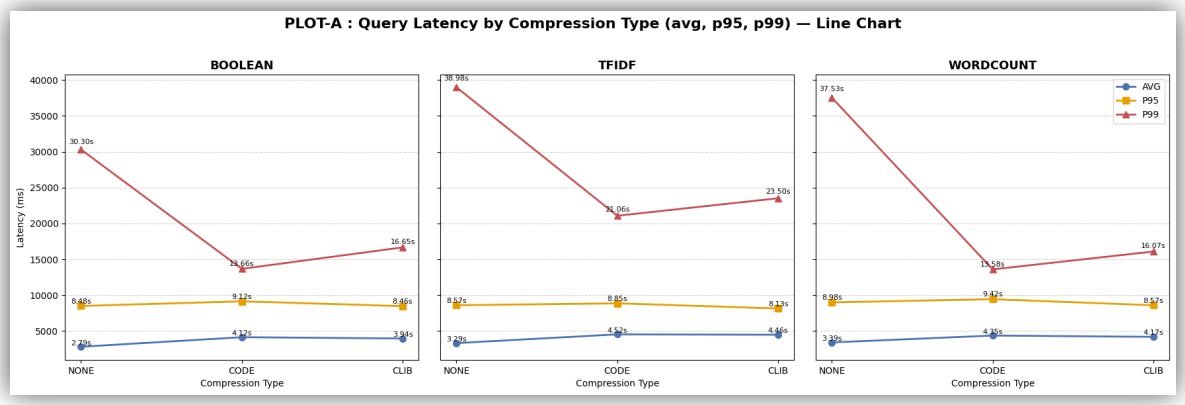

Fig v. SelfIndex – Avg, P95, P99 Query Latency vs Compression Type – Line chart

- Uncompressed data (NONE) has the lowest average latency across all retrieval methods, meaning queries respond faster on average.
- Compressed data (CLIB, CODE) shows slightly higher average times due to decompression overhead.
- Uncompressed (NONE) configurations show much higher p99 values, indicating more performance spikes or inconsistency. Some queries take much longer than average.
- Compressed (CLIB, CODE) configurations, while slower on average, have more stable tail latencies, meaning fewer extreme slow queries.
- BOOLEAN remains the fastest overall, followed by WORDCOUNT, then TFIDF (the slowest). It is consistent with their increasing computational complexity.
- There's a trade-off: uncompressed indices give faster average responses but less predictable performance, while compressed ones are slightly slower but more stable and consistent under heavy or varied query loads.

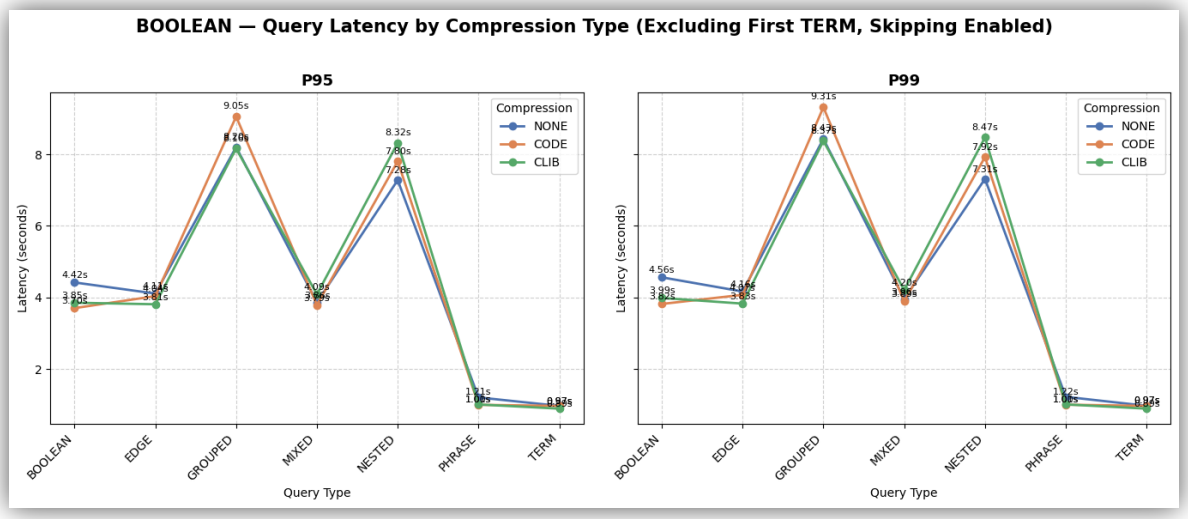

Fig w. SelfIndex – Boolean - P95, P99 Query Latency vs Compression Type

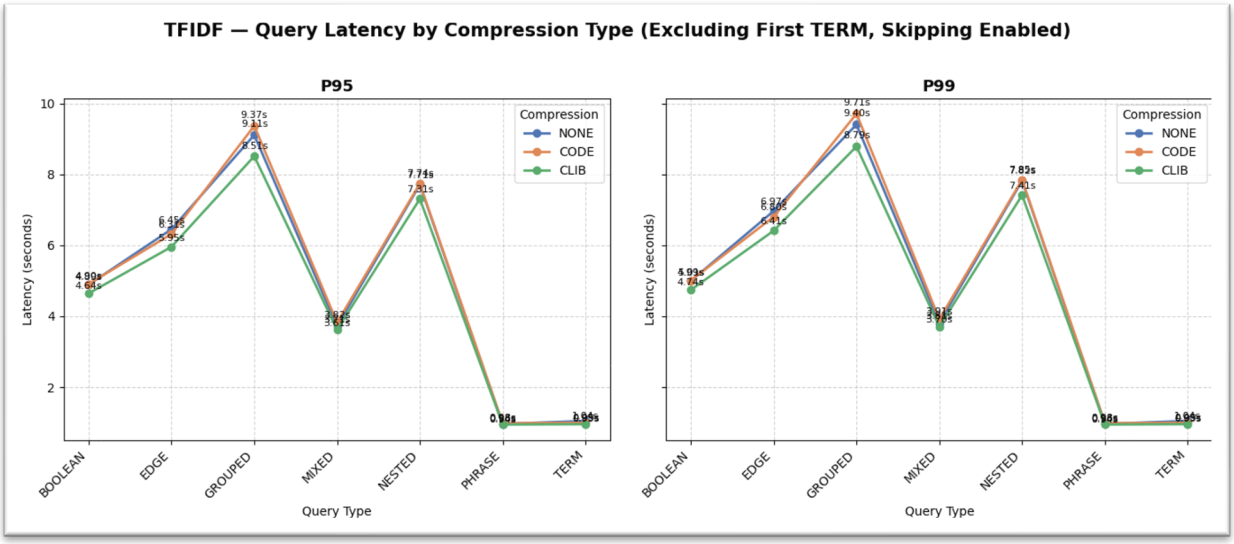

Fig x. SelfIndex – TFIDF - P95, P99 Query Latency vs Compression Type

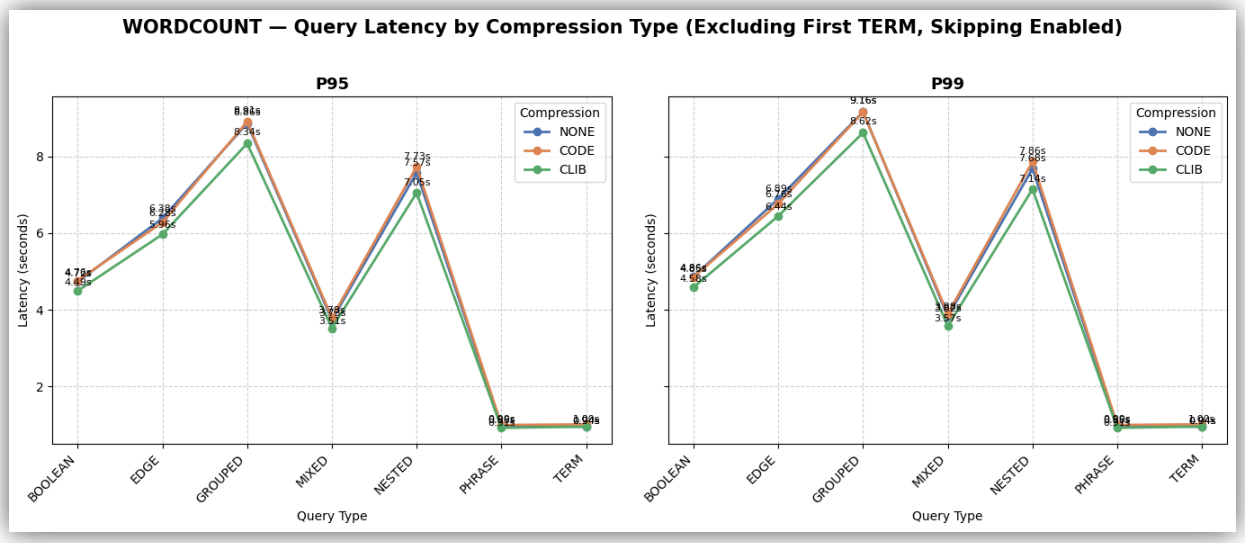

Fig y. SelfIndex – WORDCOUNT - P95, P99 Query Latency vs Compression Type

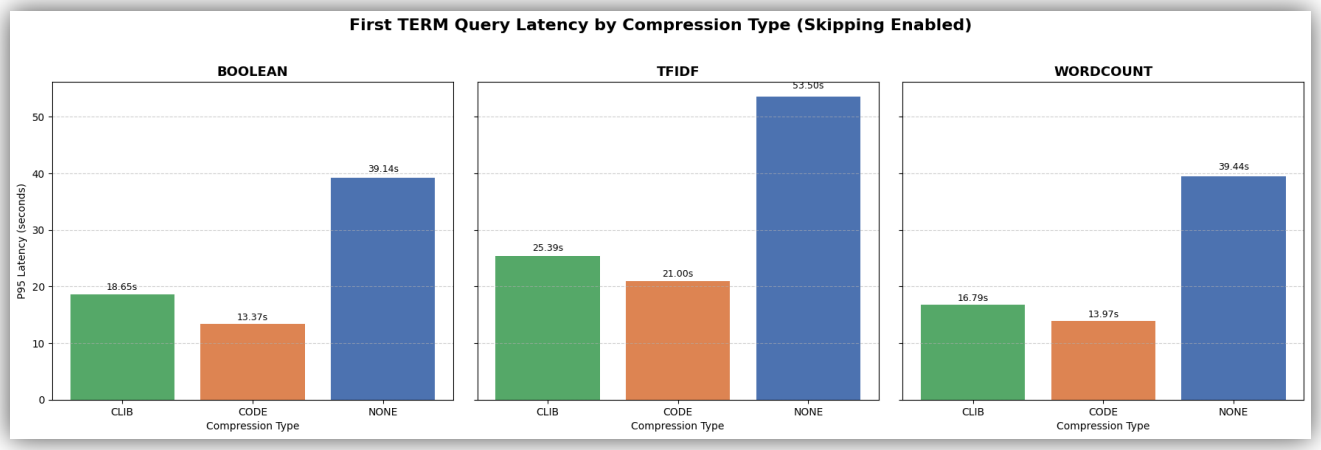

Fig z. SelfIndex – Query Latency vs Compression Type – For First term

- Uncompressed (NONE) data delivers the fastest average response times but shows higher latency variability at p95 and p99.
- Compressed types (CLIB, CODE) are slightly slower on average yet provide more consistent performance across queries.
- BOOLEAN queries are the fastest, while TFIDF is the slowest due to extra ranking calculations, with WORDCOUNT in between.
- TERM and PHRASE queries execute quickly, whereas GROUPED and NESTED queries are much slower because of their complex structure.
- Overall, BOOLEAN TERM/PHRASE queries with CLIB compression achieve the best trade-off between speed and stability.

4.7. Plot AC: Comparison of Query Processing Strategies — Term-at-a-Time (q = Tn) vs Document-at-a-Time (q = Dn) with Applied Optimizations

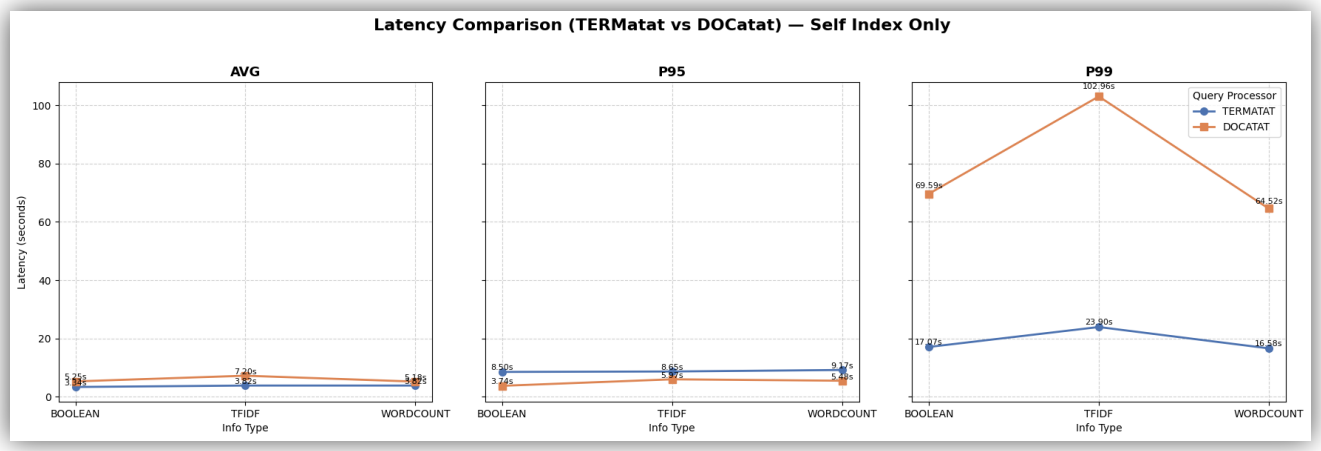

Fig (i). SelfIndex - Latency Comparition – TERMatat – DOCatat

- TERMATAT shows lower average latency across all retrieval methods, indicating faster overall query execution.
- DOCATAT has higher average and p99 latencies, meaning it's slower and less consistent under heavy loads.
- Among retrieval methods, BOOLEAN remains the fastest, while TFIDF is the slowest due to additional scoring overhead.

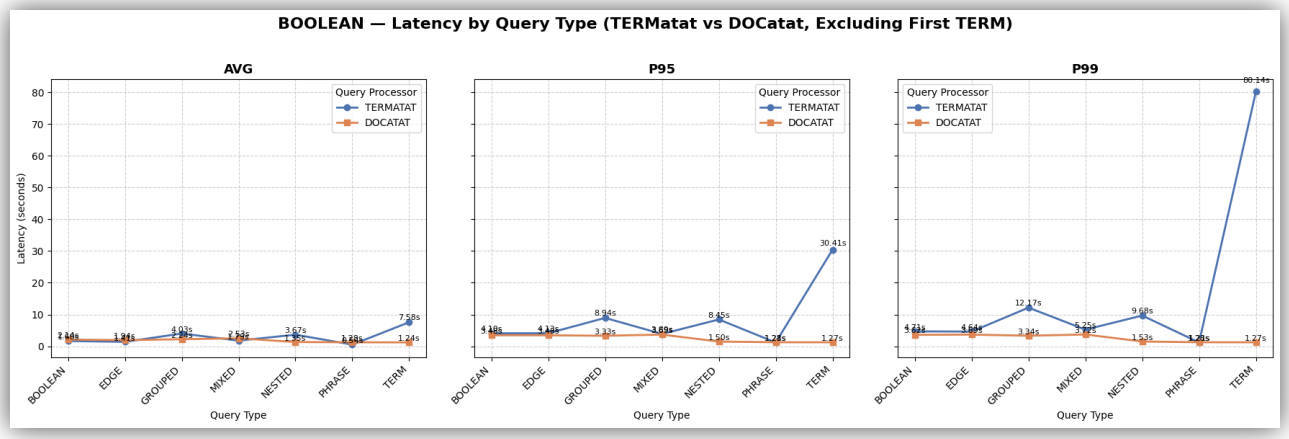

Fig (ii). SelfIndex – Boolean - Latency Comparition – TERMatat – DOCatat

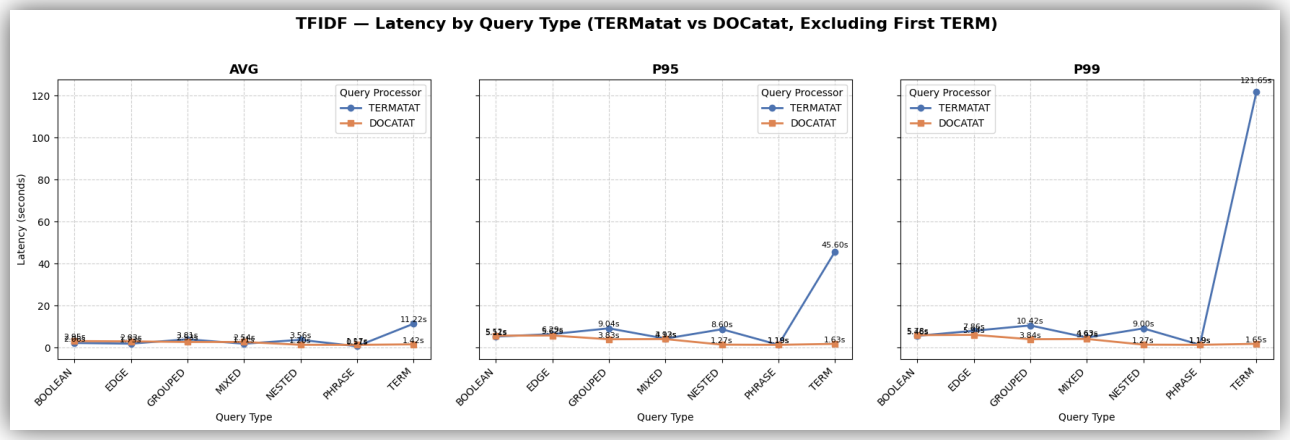

Fig (iii). SelfIndex – TFIDF - Latency Comparition – TERMatat – DOCatat

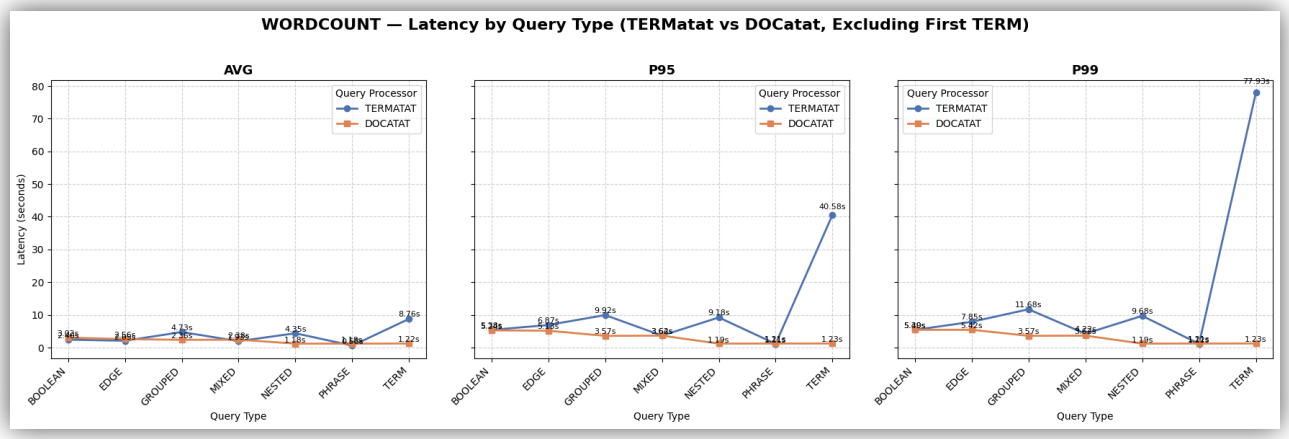

Fig (iv). SelfIndex – Wordcount - Latency Comparition – TERMatat – DOCatat

- DOCATAT consistently performs faster than TERMATAT across all info types — indicating more efficient query execution and indexing strategy.
- TERM queries under TERMATAT show extremely high latency (e.g., TFIDF TERM = 11,220 ms avg, 121,650 ms p99), likely due to complex token or posting list operations.

- PHRASE and TERM queries under DOCATAT have the lowest latencies (≈1200–1400 ms), suggesting that DOCATAT handles direct or sequential lookups efficiently.
- GROUPED and NESTED queries are significantly slower under TERMATAT (e.g., BOOLEAN GROUPED = 4,028 ms → 12,174 ms p99), implying higher aggregation or nested scan overheads.
- TFIDF and WORDCOUNT show similar trends — TERMATAT degrades latency by roughly 2–4× across most query types compared to DOCATAT.
- p95 and p99 latencies for TERMATAT are much larger, indicating more tail latency spikes — poor consistency compared to DOCATAT.
- Overall: DOCATAT offers better stability and faster responses, while TERMATAT, though more feature-rich, introduces significant latency overhead, especially for TERM and GROUPED queries.

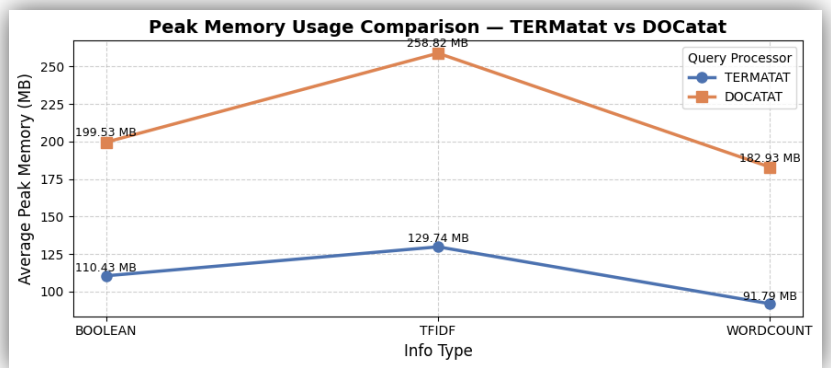

Fig (v). SelfIndex – Peak Memory Usage– TERMatat – DOCatat

- DOCATAT consumes more peak memory than TERMATAT across all **info** types.
- TFIDF uses the most memory overall, with 258.82 MB (DOCATAT) and 129.73 MB (TERMATAT), indicating heavier computation and feature storage.
- WORDCOUNT requires the least memory, especially under TERMATAT (≈91.79 MB), suggesting simpler data structures or fewer intermediate computations.
- BOOLEAN queries are moderate in memory usage between TFIDF and WORDCOUNT, consistent across both query processors.
- Overall trend: DOCATAT trades higher memory consumption for faster performance, while TERMATAT is more memory-efficient but slower.

5. Conclusion

The experiments highlight how internal design choices in an Information Retrieval (IR) system impact performance across latency, throughput, and memory efficiency. The custom-built **SelfIndex** demonstrates that:

Index structure (x) : Incorporating ranking mechanisms like WordCount and TF-IDF improves retrieval effectiveness but increases indexing overhead.

Datastore (y) : Off-the-shelf solutions such as **Redis** and **LMDB** offer faster access times and better scalability compared to custom JSON storage.

Compression (z) : Techniques like Delta encoding and Snappy reduce storage size significantly with minimal latency trade-off.

Optimizations (i) : Skip pointers provide noticeable gains in query latency, especially for large postings lists.

Query processing (q) : Document-at-a-Time (DAA) performs more efficiently for conjunctive queries, while Term-at-a-Time (TAA) is better suited for disjunctive or ranked retrieval.